

STUDY-AGILE-DevOps.md

Il Manuale Completo per Consulente Agile & DevOps per PMI

Track: AGILE — Agile Transformation, DevOps e Continuous Delivery per PMI

Autore: Elios Scoglio — per uso personale, studio e insegnamento

Versione: 1.0 — Maggio 2026

Tono: Quello di chi ha sopravvissuto a abbastanza "daily standup di 45 minuti" da avere cicatrici.

"Agile non significa fare le cose velocemente. Significa scoprire velocemente quando stai facendo la cosa sbagliata." — Una verità che metà dei team Scrum ignorano attivamente.

Come usare questo manuale

Questo manuale è per chi vuole aiutare le PMI ad adottare pratiche Agile e DevOps in modo pragmatico, non dogmatico. Non è il manuale del "fai esattamente come dice il Scrum Guide". È il manuale del "capisci il problema, adatta il metodo". Ogni parte: teoria applicata, personaggi inventati, trappole, e domande di verifica.

PARTE 1 — Agile per PMI: cosa funziona davvero (e cosa è teatro)

Il concetto

Agile non è una metodologia. È un insieme di valori e principi (Agile Manifesto, 2001) che guidano come si lavora. I framework (Scrum, Kanban, SAgile, XP) sono implementazioni specifiche, non l'Agile stesso.

I 4 valori del Manifesto:

- Individui e interazioni > processi e strumenti
- Software funzionante > documentazione esaustiva
- Collaborazione con il cliente > negoziazione del contratto
- Rispondere al cambiamento > seguire un piano

Cosa funziona per le PMI:

- Sprint di 1-2 settimane con deliverable concreti
- Daily standup di 15 minuti massimo (non status meeting)
- Retrospective ogni 2 settimane per migliorare il processo
- Backlog prioritizzato con criteri di business espliciti
- Definition of Done chiara e rispettata da tutti
- Feedback loop rapidi con il cliente/stakeholder

Cosa è teatro Agile (da evitare):

- Daily standup di 45 minuti dove tutti riportano al manager
- Scrum senza sprint review (chi controlla la qualità?)
- Retrospective dove si elencano problemi senza azioni

- Backlog con 300 user story non priorizzate
- Velocity come KPI principale (è un numero interno, non un valore)
- "Siamo Agile" mentre il deploy avviene ogni 3 mesi

Perché conta

Il 70% delle "trasformazioni Agile" produce Agile theatre: le parole cambiano (sprint, backlog, standup) ma i comportamenti rimangono gli stessi (waterfall con più meeting). Il tuo valore è aiutare le PMI a capire cosa Agile risolve davvero: cicli di feedback più rapidi, riduzione del rischio di costruire la cosa sbagliata, e team più autonomi e responsabili.

Esempio

Il Daily Standup di 45 minuti di TechBridge

TechBridge aveva introdotto "Scrum" 6 mesi prima. Il loro daily standup durava 45 minuti. Perché? Perché ogni persona spiegava in dettaglio tutto quello che aveva fatto il giorno precedente, incluse le difficoltà tecniche, le email ricevute, e i meeting a cui aveva partecipato. Il project manager usava il tempo per dare nuovi task. Nessuno aveva mai letto il Scrum Guide.

La tua diagnosi: il daily standup è stato trasformato in un meeting di status report al management. Non serve a sincronizzare il team: serve a "controllare" il team. Questo è l'opposto di Agile. L'intervento: facilitare 3 daily standup mostrando il formato corretto (3 domande: cosa ho fatto ieri, cosa faccio oggi, ho blocchi?), 15 minuti massimo, nessun management report. Dopo 2 settimane: il team stesso difende il formato corretto.

Alternativa

Non tutti i team hanno bisogno di Scrum. Per team con flusso continuo e priorità imprevedibili (es. supporto tecnico, manutenzione), Kanban è spesso più adatto: board visuale, WIP limits, focus sul flusso invece che sugli sprint. Nessun commitment temporale fisso, ma visualizzazione delle code e blocchi.

Cosa NON fare

Non proporre "facciamo SAFe" a una PMI da 20 persone. SAFe (Scaled Agile Framework) è progettato per organizzazioni di 500+ persone con 5+ team. Per una PMI è un cannone per ammazzare una mosca: overhead enorme, zero beneficio. La complessità del framework deve essere proporzionale alla complessità del problema.

****Attenzione!**** L'"Agile Coach" che arriva con le slide di Scrum e fa seguire il framework alla lettera senza capire il contesto specifico del team non è un consulente: è un venditore di framework. Il tuo valore è adattare il metodo al contesto, non applicare il metodo indipendentemente dal contesto.

****Tip da campo**** La prima domanda in ogni engagement Agile: "Qual è il problema che volete risolvere con Agile?" Se la risposta è "ce lo ha detto il CEO", hai un problema di commitment. Se la risposta è "abbiamo progetti che finiscono sempre in ritardo e fuori scope", hai un problema reale da risolvere.

Domande di verifica:

- Quali sono i 3 segnali più comuni di "Agile theatre"?
- Come distingueresti Scrum da Kanban e quando sceglieresti l'uno vs l'altro?
- Come risponderesti a un CEO che dice "vogliamo fare SAFe perché lo fa Amazon"?

PARTE 2 — Sprint Planning: costruire il giusto, non velocemente

Il concetto

Lo Sprint Planning è il meeting in cui il team decide cosa farà nello sprint (1-2 settimane). Fatto bene, produce un obiettivo di sprint chiaro e un backlog di sprint credibile. Fatto male, produce un elenco di task troppo lungo che il team "sistema" durante lo sprint.

Struttura Sprint Planning efficace (2-4 ore per sprint di 2 settimane):

Parte 1 — What (60-90 min):

- Product Owner presenta le user story priorizzate
- Team fa domande di chiarimento (non stima ancora)
- Team e PO concordano l'obiettivo dello sprint
- Team seleziona le user story che rientrano nella capacity

Parte 2 — How (60-90 min):

- Per ogni user story selezionata, il team scompone in task tecnici
- Stima dei task (se necessario per capire la fattibilità)
- Identificazione delle dipendenze e dei rischi

Output:

- Sprint Goal: un obiettivo in una frase ("Questo sprint, il customer potrà completare l'acquisto senza errori sul pagamento")
- Sprint Backlog: lista di user story selezionate con task tecnici chiari
- Commitment: il team si impegna pubblicamente sul backlog

Buone pratiche:

- Nessuna user story entra in sprint senza Definition of Ready (comprensibile, stimabile, dipendenze identificate)
- Sprint Goal prima del backlog: il goal guida la selezione, non il contrario
- Capacity realistica: sottrai tempo per meeting, 1:1, sick days, code review
- Mai accettare story a metà sprint (rompe il focus e la prevedibilità)

Perché conta

Uno sprint planning fatto male produce sprint caotici, commitment violati, e team frustrato. Uno fatto bene produce focus, prevedibilità, e team che sa perché sta facendo quello che fa.

Esempio

Lo Sprint Planning di NovaTech che durava 4 ore e produceva poco

NovaTech aveva sprint planning di 4 ore in cui il team discuteva ogni dettaglio tecnico di ogni user story. Alla fine, il backlog era pieno ma il team era esausto e molte story erano ancora vaghe. Il problema: stavano mescolando "what" (cosa facciamo) e "how" (come lo facciamo) in modo caotico, e le user story non avevano Definition of Ready.

L'intervento: introduzione della Definition of Ready (ogni story deve avere: titolo, descrizione utente, criteri di accettazione, mockup se necessario, dipendenze identificate). Le story che non rispettano la DoR non entrano in sprint planning. Risultato: meeting sceso a 2 ore, backlog più credibile, sprint più prevedibili.

Alternativa

Per team molto junior o nuovi ad Agile, il planning poker (tecnica di stima consensuale con carte Fibonacci) aiuta a calibrare la capacity e a identificare le incomprensioni nascoste. Quando un developer dice "1 story point" e un altro dice "13", c'è qualcosa da chiarire sulla story.

Cosa NON fare

Non usare la velocity come target ("dobbiamo fare 40 SP questo sprint"). La velocity è un indicatore storico per la pianificazione, non un obiettivo da ottimizzare. Un team che ottimizza la velocity invece di ottimizzare il valore per il cliente impara a spaccare le story in piccoli pezzi e a gonfiare le stime.

****Attenzione!**** Il "commitment" dello sprint non è un contratto. È una previsione basata sulle conoscenze attuali. Se durante lo sprint emerge un blocco imprevedibile (dipendenza esterna, scope più complesso del previsto), il team deve poter rinegoziare con il PO invece di lavorare di notte per "mantenere il commitment". Il commitment è onestà, non eroismo.

****Tip da campo**** Il Sprint Goal è la parte più negletta del planning. "Fare le 8 story del backlog" non è un goal: è una lista. Un goal descrive il valore per l'utente o il business: "Questo sprint il processo di checkout sarà completamente funzionante per il 90% dei percorsi utente". Quando il team ha un goal chiaro, sa come prendere decisioni durante lo sprint senza chiedere al PO ogni 2 ore.

Domande di verifica:

- Cosa deve avere una user story per essere "Definition of Ready"?
- Come gestiresti il caso in cui a metà sprint il PO vuole aggiungere una story urgente?
- Qual è la differenza tra sprint planning e sprint goal?

PARTE 3 — Retrospettiva: il motore del miglioramento continuo

Il concetto

La retrospettiva è il meeting più importante dell'Agile. È dove il team impara. Se saltata o fatta male, il team ripete gli stessi errori sprint dopo sprint.

Struttura base (60-90 min, ogni sprint):

Check-in (5 min): Una parola che descrive come ti senti su questo sprint

Data collection (15 min): Cosa è andato bene (Keep), cosa non ha funzionato (Drop), cosa vogliamo provare (Try). Ogni persona scrive su post-it, poi si condivide.

Insights (20 min): Raggruppamento per temi. Votazione sui temi prioritari (dot voting: ogni persona ha 3 punti da assegnare).

Actions (20 min): Per i top 2-3 temi: quali azioni concrete, chi le prende in carico, quando le verifichiamo. Massimo 3 azioni per retrospettiva.

Chiusura (5 min): Come è stata la retrospettiva? (1-5 stelle, feedback sul processo)

Formati alternativi:

- 4Ls: Liked, Learned, Lacked, Longed For
- Mad/Sad/Glad: emozioni come entry point
- Starfish: Stop, Less, Keep, More, Start
- Timeline: ripercorrere gli eventi dello sprint cronologicamente
- Futurespective: cosa vorremmo che fosse vero tra 6 mesi?

Perché conta

Una retrospettiva efficace produce 2-3 azioni concrete che il team implementa nello sprint successivo. Dopo 6 sprint di buone retrospettive, il team ha migliorato 12-18 aspetti del suo processo. Questo è il compounding del miglioramento

continuo.

Esempio

La retrospettiva "lista di lamentele" di WebGroup

WebGroup faceva retrospettive ogni 2 settimane. Duravano 45 minuti e producevano una lista di 10-15 problemi. Nessuna azione concreta. Alla retrospettiva successiva, la stessa lista con qualche problema nuovo aggiunto. Il team si sentiva inascoltato: "tanto non cambia niente". Il facilitatore (esterno, prima di te) non sapeva gestire la dinamica.

Il tuo intervento: retrospettive con massimo 3 azioni, owner esplicito, verifica alla retrospettiva successiva. Dopo 3 sprint: il team inizia a vedere i cambiamenti reali, la fiducia nel processo cresce, e le retrospettive diventano meno "sfogo" e più "miglioramento".

Cosa NON fare

Non fare sempre la stessa struttura (Keep/Drop/Try ogni sprint). La routine uccide la riflessione. Varia il formato ogni 2-3 sprint. La variazione mantiene l'engagement e fa emergere insight diversi.

****Attenzione!**** La retrospettiva deve essere uno spazio sicuro. Se il manager usa le informazioni emerse in retrospettiva per valutazioni individuali, il team smette di essere onesto. La retrospettiva è per il team, non per il management.

****Tip da campo**** Inizia ogni retrospettiva con "Assumiamo che tutti abbiano fatto del loro meglio con le informazioni e le risorse che avevano." (Norm Kerth's Prime Directive). Questo frame abbassa la difensività e focalizza la conversazione sui sistemi, non sulle persone.

Domande di verifica:

- Come gestiresti una retrospettiva in cui il team è silenzioso e nessuno vuole parlare?
- Come garantiresti che le azioni di retrospettiva vengano davvero implementate?
- Qual è il segnale che una retrospettiva è "teatro" invece che reale miglioramento?

PARTE 4 — DevOps: la cultura oltre gli strumenti

Il concetto

DevOps non è un ruolo ("il DevOps"). Non è uno strumento (Docker, Jenkins, Kubernetes). È una cultura e un insieme di pratiche che eliminano il muro tra chi sviluppa (Dev) e chi gestisce l'infrastruttura (Ops).

I 3 principi DevOps (The Phoenix Project):

1. Flow: ottimizza il flusso dal commit alla produzione. Identifica e rimuovi i colli di bottiglia. Visualizza il flusso con Value Stream Mapping.
2. Feedback: crea feedback loop rapidi ad ogni stage. I test falliti devono arrivare al developer in minuti, non giorni. I problemi in produzione devono essere visibili immediatamente.
3. Continuous Learning: impara dai fallimenti (post-mortem), sperimenta in sicurezza (feature flags, canary), diffonde la conoscenza (documentazione, runbook, ChatOps).

Le pratiche DevOps fondamentali:

- Infrastructure as Code (Terraform, Ansible)

- Continuous Integration (ogni commit triggera build + test)
- Continuous Delivery (ogni commit può andare in produzione)
- Containerizzazione (Docker, Kubernetes)
- Monitoring e Alerting (Prometheus, Grafana, PagerDuty)
- Incident Management (runbook, SLA, post-mortem)

Perché conta

La separazione Dev/Ops produce sistemi che "funzionano sulla macchina dello sviluppatore ma non in produzione", deployment che richiedono 2 giorni di coordinamento, e incidenti che nessuno sa come gestire. DevOps riduce il time to market, aumenta la stabilità, e responsabilizza il team sulla salute del sistema in produzione.

Esempio

Il "works on my machine" di AppDev

AppDev aveva 8 developer e un "sistemista" (Maurizio) che gestiva i 3 server di produzione da solo, senza documentazione. Ogni deployment richiedeva un ticket a Maurizio che lo gestiva "quando aveva tempo" (2-3 giorni). Se c'era un problema in produzione, Maurizio era l'unico che sapeva come intervenire. Quando Maurizio è andato in ferie 2 settimane, si sono fermati.

L'intervento: Infrastructure as Code (Ansible per il provisioning), Docker per i container applicativi, pipeline CI/CD su GitLab. Dopo 3 mesi: i developer possono deployare in autonomia, Maurizio è stato "de-heroized" (non è più l'unico punto di conoscenza), e il team ha finalmente un runbook. Maurizio è passato da firefighter a infrastructure architect.

Alternativa

Per PMI che non hanno budget per un DevOps engineer, i servizi cloud managed riducono enormemente la complessità operativa: AWS ECS/Fargate, Railway, Render, Fly.io permettono deployment containerizzati senza gestire Kubernetes. Non è la soluzione enterprise, ma per una PMI da 5-20 developer è sufficiente.

Cosa NON fare

Non iniziare da Kubernetes. Kubernetes è potente ma richiede una curva di apprendimento ripida e un team dedicato alla gestione. Per le PMI, inizia con Docker + CI/CD + un container platform managed. Introduci Kubernetes solo quando hai team che possono gestirlo.

****Attenzione!**** "DevOps Engineer" come unico ruolo è un anti-pattern. DevOps è una cultura condivisa tra tutti i developer, non la responsabilità di una singola persona. Se hai un "DevOps" che fa tutto da solo mentre i developer "non si occupano di infrastruttura", hai DevOps nel nome ma non nella pratica.

****Tip da campo**** Il primo passo per introdurre DevOps in un team è rendere visibile il flusso. Fai un Value Stream Map: disegna tutti i passi dal "sviluppatore completa la feature" al "l'utente usa la feature in produzione". Quanto tempo ci vuole? Dove sono i tempi di attesa? Spesso il tempo di attesa è il 90% del totale, il lavoro effettivo il 10%.

Domande di verifica:

- Qual è la differenza tra Continuous Integration, Continuous Delivery e Continuous Deployment?
- Come implementeresti Infrastructure as Code in un'azienda che non l'ha mai usata?
- Cosa includeresti in un runbook per un incidente in produzione?

PARTE 5 — User Story e Product Backlog: il cuore dell'Agile

Il concetto

Una User Story è una descrizione del valore per l'utente, non una specifica tecnica. Il formato standard:

```
Come [chi sono],
voglio [cosa fare],
per [quale beneficio].
```

Buona user story: "Come cliente registrato, voglio salvare il mio metodo di pagamento preferito, per non doverlo reinserire ad ogni acquisto."

Cattiva user story: "Implementare tabella user_payments con FK user_id."

I criteri INVEST per una buona user story:

- Independent: non dipende da un'altra story per essere implementata
- Negotiable: il team e il PO possono discutere l'implementazione
- Valuable: porta valore all'utente o al business
- Estimable: il team sa stimarla
- Small: completabile in uno sprint
- Testable: si può scrivere un test che verifica se è soddisfatta

Product Backlog sano:

- Priorizzato per valore di business (non per urgenza tecnica)
- I top 10-15 item sono "Ready" (pronti per lo sprint)
- Non ha più di 50-80 item totali (oltre è un graveyard)
- Viene "groomed" (raffinato) ogni settimana

Perché conta

Un backlog mal gestito è una fonte infinita di frustrazione: item che entrano in sprint senza essere capiti, priorità cambiate ogni giorno, user story che sono in realtà bug mascherati, e scope creep continuo. Un backlog ben gestito è la priorità più importante del Product Owner.

Esempio

Il Backlog da 400 item di FinDev

FinDev aveva un backlog con 400 user story. Il top 5 era invariato da 3 mesi (troppo complesso per entrare in sprint). Il fondo del backlog aveva item del 2019 che nessuno ricordava più. Il PO aggiungeva nuovi item ogni giorno senza rimuovere niente. Il team non sapeva mai cosa sarebbe successo nel prossimo sprint.

Il tuo intervento: Backlog Pruning Session — 3 ore con il team e il PO. Regola: ogni item > 6 mesi non prioritizzato viene archiviato (non cancellato, archiviato). Ogni item > 3 mesi in "top 20" che non è ancora "Ready" viene spezzato o riscritto. Risultato: backlog da 400 a 60 item, top 15 tutti "Ready", PO che finalmente ha un tool di gestione delle priorità, non un deposito di wishlist.

Cosa NON fare

Non confondere user story con task tecnici. "Refactoring del modulo X" non è una user story: è un enabler tecnico. Può stare nel backlog come technical story (con un business rationale esplicito: "questo riduce il tempo di deploy del 50%"), ma non come user story.

****Attenzione!**** Il "grooming" del backlog non è un meeting opzionale. È la preparazione per il planning. Se le story entrano in planning già chiare, accettate, e stimate, il planning prende 1 ora. Se entrano grezzo, prende 4 ore. Il tempo di preparazione si risparmia nel meeting.

****Tip da campo**** I criteri di accettazione di una user story sono il contratto tra PO e team. Devono essere: specifici ("il sistema invia un'email di conferma"), verificabili ("il cliente riceve l'email entro 30 secondi"), e scritti dal PO prima che il team stimi. Se il team stima senza criteri di accettazione, sta stimando alla cieca.

Domande di verifica:

- Riscrivi questa user story in formato INVEST: "Come sviluppatore, voglio che il database sia ottimizzato."
- Come gestiresti un PO che continuamente cambia le priorità nel mezzo dello sprint?
- Qual è la differenza tra "Definition of Ready" e "Definition of Done"?

PARTE 6 — Kanban: il flusso come primo principio

Il concetto

Kanban è un sistema per gestire il flusso di lavoro in modo visuale. I principi:

- Visualizzare il flusso (board con colonne: To Do, In Progress, Done)
- Limitare il Work In Progress (WIP limits per colonna)
- Gestire il flusso (monitorare il cycle time, identificare i blocchi)
- Rendere esplicite le policy (criteri per spostare un item)
- Implementare feedback loop
- Migliorare collaborativamente

WIP Limits: il principio più importante e più ignorato. Limitare il numero di item "In Progress" contemporaneamente forza il team a finire le cose prima di iniziarne di nuove (invece di avere 10 cose "in corso" e 0 completate).

Metriche Kanban:

- Cycle Time: dal "iniziato" al "done" per un singolo item
- Lead Time: dal "richiesto" al "done" (include l'attesa in backlog)
- Throughput: quanti item completati per unità di tempo
- Aging Work: item "in progress" da troppo tempo (segnale di blocco)

Perché conta

I team che non hanno WIP limits tipicamente hanno 5-10 task "in corso" contemporaneamente per persona. Nessuno è finito, tutto è "quasi finito". I WIP limits forzano una domanda scomoda: "perché questo task è bloccato?" e stimolano la collaborazione per sbloccare invece di prendere nuovo lavoro.

Esempio

Il WIP limit che ha cambiato il team di SupportPro

SupportPro aveva un team di supporto tecnico con una board Kanban senza WIP limits. Ogni giorno, 15-20 ticket erano "In Progress". Nessuno finiva mai nulla. I ticket rimanevano aperti per giorni perché appena qualcuno si bloccava, prendeva un nuovo ticket invece di sbloccare quello precedente.

WIP limit introdotto: max 3 ticket per persona "In Progress". Effetto immediato: le persone non potevano prendere nuovi ticket. Effetto a cascata: il team iniziò a collaborare per sbloccarsi ("ho un ticket bloccato perché aspetto risposta da cliente, puoi aiutarmi a sbloccare il tuo invece di prenderne uno nuovo?"). Cycle time medio sceso da 5 giorni a 1.8 giorni in 3 settimane.

Alternativa

Per team che devono gestire sia feature development (sprint-based) che manutenzione/supporto (flusso continuo), una

board ibrida funziona: sprint board per le feature, Kanban board per bug e supporto. Le due board hanno SLA diversi e WIP limits diversi.

Cosa NON fare

Non introdurre Kanban come "Scrum senza sprint". Kanban e Scrum hanno filosofie diverse: Scrum usa iterazioni temporali per creare ritmo e prevedibilità, Kanban usa il flusso continuo per massimizzare la velocità. Sceglierne uno basandoti sulla natura del lavoro, non sulla popolarità.

****Attenzione!**** Una board Kanban con 20 colonne non è Kanban: è burocrazia visualizzata. Le colonne devono corrispondere agli stati reali del lavoro (non ai reparti dell'azienda). Inizia con 3-4 colonne e aggiungi solo se hai evidenza che uno stato aggiuntivo è necessario.

****Tip da campo**** Il **Cumulative Flow Diagram** (CFD) è la metrica più potente del Kanban. Se le bande si allargano (gli item si accumulano in una fase), hai un bottleneck lì. Se si restringono, stai migliorando. La matematica del flusso non mente.

Domande di verifica:

- Qual è il vantaggio principale del WIP limit? Perché molti team resistono ad implementarlo?
- Come costruiresti una board Kanban per un team di sviluppo che gestisce sia feature che bug?
- Cosa indica un "aging work item" nella tua board?

PARTE 7 — Definition of Done: la qualità non è un'opinione

Il concetto

La Definition of Done (DoD) è l'accordo del team su cosa significa che un lavoro è "completato". Non "il codice funziona", ma tutti i criteri di qualità che un item deve soddisfare prima di essere considerato finito.

Una DoD tipica per un team di sviluppo:

- [] Codice scritto e funzionante
- [] Unit test scritti e passanti (coverage > X%)
- [] Integration test passanti
- [] Code review approvata da almeno 1 peer
- [] Nessun bug bloccante
- [] Documentazione aggiornata (se API o comportamento pubblico)
- [] Deployato in staging
- [] Smoke test passante in staging
- [] Criteri di accettazione verificati dal PO
- [] Nessun secret in codice
- [] Log strutturati presenti per i flussi critici

Livelli di DoD:

- DoD dell'item: criteri per una singola user story
- DoD dello sprint: criteri aggiuntivi per un'iterazione (es. release candidate)
- DoD della release: criteri per il deployment in produzione

Perché conta

Senza DoD, "done" significa cose diverse per persone diverse. Il developer dice "ho finito" intendendo "il codice compila". Il PO lo mette in sprint review. Il cliente lo trova pieno di bug. La DoD è il contratto di qualità del team con sé

stesso.

Esempio

La Discovery della DoD Mancante in DevMax

DevMax aveva una sprint velocity di 60 story point a sprint. Sembrava ottimo. Ma ogni sprint aveva 5-8 bug critici trovati in staging post-sprint. La causa: nessun test in staging come criterio di "done". Il developer completava il codice, passava la review, e il ticket era "done". Nessuno lo testava in staging prima della sprint review.

Aggiunta alla DoD: "Deployato in staging e smoke test passante". Velocity immediatamente scesa a 40 SP. Tutti spaventati. Ma il numero di bug critici sceso da 7 a 1 per sprint. Il software ora era davvero "done". La velocity apparente era gonfiata da item che non erano davvero finiti.

Cosa NON fare

Non fare la DoD una lista di 30 criteri impossibili da rispettare. La DoD deve essere ambiziosa ma raggiungibile. Se il team viola sistematicamente 3 criteri della DoD, quei criteri non sono nella DoD: sono aspirazioni. Meglio una DoD di 8 criteri sempre rispettati che una da 20 criteri rispettati al 60%.

****Attenzione!**** La DoD si evolve nel tempo. All'inizio può essere semplice (codice + review). Man mano che il team matura, si aggiungono criteri (test automatici, monitoring, documentation). Revedi la DoD ogni 2-3 mesi in retrospettiva.

****Tip da campo**** La DoD non si impone: si costruisce insieme. Fai una sessione di 30-45 minuti con il team: "Cosa deve essere vero per poter dire che questo lavoro è davvero finito?" Le risposte del team sono molto più credibili dei criteri imposti dall'esterno.

Domande di verifica:

- Come costruiresti la Definition of Done con un team che non l'ha mai avuta?
- Cosa fai quando la DoD rallenta troppo il team?
- Qual è la differenza tra DoD e criteri di accettazione di una user story?

PARTE 8 — Agile Metrics: misurare ciò che conta

Il concetto

Le metriche Agile servono per migliorare il processo, non per controllare le persone. Le metriche usate male distruggono la cultura e producono gaming (ottimizzazione della metrica a scapito del valore reale).

Metriche utili:

Velocity — story point completati per sprint. Utile per la previsione ("in quanti sprint completeremo questo backlog?"). Inutile come obiettivo o confronto tra team.

Cycle Time — tempo dal "iniziato" al "done" per un singolo item. Indica la prevedibilità del processo. Vuoi distribuzioni tight, non wide.

Lead Time — tempo dal "richiesto" al "done". Include l'attesa. Riflette la risposta del team alle richieste del business.

Defect Rate — numero di bug trovati per sprint (in testing, in staging, in produzione). Trend in calo = qualità che migliora.

Team Happiness — survey mensile (1-5) su motivazione, collaborazione, chiarezza degli obiettivi. Predittivo di future performance e retention.

Deployment Frequency — quante volte il team deploia in produzione. DORA metric chiave: team elite deployano multiple volte al giorno.

MTTR (Mean Time to Restore) — tempo medio per ripristinare il servizio dopo un incidente. Misura la resilienza operativa.

Metriche da NON usare:

- Linee di codice scritte
- Ore lavorate
- Numero di commit
- Story point per persona
- Velocity come competizione tra sprint o team

Perché conta

Le metriche sbagliate producono comportamenti sbagliati. Un team misurato sulle linee di codice scriverà codice prolisso. Un team misurato sulla velocity gonfierà le stime. Le metriche giuste creano i comportamenti giusti.

Esempio

Il Velocity Game di PlatformHub

PlatformHub misurava la velocity del team e la mostrava al management settimanalmente. Il team capì il gioco: le stime delle user story salirono del 30% in 3 sprint ("sicurezza" per mantenere la velocity alta). Il management era felice della "alta velocità". Il throughput reale (valore consegnato) era in calo. La velocity sembrava stabile ma il progresso reale rallentava.

Quando tu hai introdotto il Cycle Time come metrica primaria (al posto della velocity), il gaming è diventato impossibile: il cycle time si misura dai ticket, non dalle stime. In 2 mesi il team ha smesso di gonfiare le stime e si è concentrato a ridurre il cycle time reale.

Cosa NON fare

Non presentare le metriche al management come "lo stato di salute del team" senza contesto. Una velocity bassa in uno sprint in cui c'era un incidente in produzione non significa che il team è poco produttivo. Il contesto è tutto.

****Tip da campo**** Le DORA metrics (Deployment Frequency, Lead Time for Changes, MTTR, Change Failure Rate) sono le quattro metriche più predittive della performance dei team di software engineering. Sono validate empiricamente su migliaia di team globali. Se il cliente vuole capire dove migliorare, parte da queste quattro.

Domande di verifica:

- Come spiegheresti a un manager perché la velocity non deve essere un obiettivo?
- Quali 3 metriche DORA proporresti come punto di partenza a un team che non misura niente?
- Come useresti il Cycle Time per identificare un bottleneck nel processo?

PARTE 9 — Il tuo prodotto AGILE: prezzi, struttura, upsell

Il concetto

I prodotti del track AGILE:

Prodotto | Prezzo | Durata | Output

Agile Health Check	800–1.500€	1 sett.	Diagnosi + priorità
Agile Transformation Sprint	3.000–6.000€	4-6 sett.	Nuove pratiche attive
Scrum/Kanban Coaching	1.000–2.000€/mese	3-6 mesi	Team facilitato e formato
DevOps Assessment	2.000–4.000€	2-3 sett.	Pipeline + cultura assessment
DevOps Implementation	4.000–10.000€	6-10 sett.	CI/CD + IaC + monitoring

Upsell naturali:

- AGILE → LEAD: il processo funziona ma le persone hanno bisogno di crescere come leader
- AGILE → ARCH: il processo migliora ma l'architettura frena la velocità di delivery
- AGILE → FCTO: hanno bisogno di governance complessiva, non solo processo

Le obiezioni AGILE:

- "Abbiamo già provato Scrum e non ha funzionato" → "Agile fallisce raramente per i principi; fallisce quasi sempre per l'implementazione. Raccontami cosa è successo."
- "I nostri progetti sono troppo imprevedibili per gli sprint" → "L'imprevedibilità è esattamente quello che Agile gestisce meglio. Kanban potrebbe essere più adatto di Scrum per voi."
- "I nostri clienti non vogliono meeting settimanali" → "La sprint review non richiede la presenza del cliente ogni sprint. Richiede un feedback loop regolare, che può essere anche asincrono."

****Tip da campo**** Il modo migliore per vendere Agile a un'azienda scettica è mostrare il costo dell'anti-agile: quante volte nell'ultimo anno hanno consegnato un progetto in ritardo o fuori scope? Quanto è costato? Il costo del "processo attuale che non funziona" è spesso molto più alto del costo del cambiamento.

PARTE 10 — CI/CD in pratica: dalla pipeline alla cultura

Il concetto

Continuous Integration (CI): ogni commit triggera automaticamente un build + test suite. Il developer ottiene feedback in minuti (non giorni) se il suo codice ha rotto qualcosa.

Continuous Delivery (CD): ogni commit che passa la CI è potenzialmente deployabile in produzione. Il deploy è una decisione di business, non tecnica.

Continuous Deployment: ogni commit che passa la CI viene deployato automaticamente in produzione. Solo per organizzazioni con maturità molto alta e test suite molto affidabili.

Una pipeline CI/CD standard:

```
Commit → Lint + Static Analysis → Unit Tests → Build →
Integration Tests → Deploy Staging → Smoke Tests →
[Manual Approval for Prod] → Deploy Production → Health Check
```

Strumenti comuni:

- GitLab CI/CD (default per team su GitLab)
- GitHub Actions (default per team su GitHub)
- Jenkins (vecchio standard, ancora molto usato)
- ArgoCD (GitOps per Kubernetes)
- Terraform Cloud (IaC pipeline)

Quality gates — criteri che bloccano la pipeline se non soddisfatti:

- Test falliti → pipeline bloccata
- Coverage < soglia → pipeline bloccata
- Security vulnerability critica → pipeline bloccata
- Docker image con CVE critico → pipeline bloccata

Perché conta

Una pipeline CI/CD ben costruita è il "sistema immunitario" del team: cattura i problemi prima che arrivino in produzione. Senza CI/CD, i problemi si scoprono tardi (in staging o produzione), quando sono molto più costosi da risolvere.

Esempio

La Pipeline da 45 Minuti di ApiFlow

ApiFlow aveva una CI che durava 45 minuti. I developer aspettavano 45 minuti per ogni feedback. Risultato: facevano commit grandi (per non aspettare spesso), batch grandi = più rischio, più difficoltà a isolare i bug.

La tua analisi: i test di integrazione erano lenti (30 min) perché avviavano un database reale per ogni test. La soluzione: parallelizzazione dei test (split su 3 worker paralleli), mock del database per i test unit, integration test su database in-memory. Risultato: pipeline da 45 min a 8 min. Commit frequency triplicata.

Cosa NON fare

Non introdurre una pipeline CI/CD complessa in un team che non ha ancora test. Prima porta il coverage a un livello minimo accettabile (30-40%), poi costruisci la pipeline. Una pipeline che non ha niente da testare è solo overhead.

****Tip da campo**** La regola del "broken window": se la pipeline è rossa, risolverla è la priorità #1, prima di qualsiasi nuova feature. Un team che tollera pipeline rosse per giorni perde fiducia nel processo e inizia a ignorare i feedback. La pipeline verde è un contratto culturale.

CHECKLIST — Agile/DevOps Consultant

Assessment iniziale

- ☐ Ho capito l'attuale processo di delivery (dal commit alla produzione)
- ☐ Ho misurato il Lead Time e il Cycle Time attuali
- ☐ Ho identificato i principali colli di bottiglia
- ☐ Ho capito la maturità del team (test, CI/CD, documentazione)
- ☐ Ho intervistato developer, PO, e stakeholder business

Transformation

- ☐ Ho costruito la DoD con il team (non imposta)
- ☐ Ho introdotto le retrospettive come meccanismo di miglioramento
- ☐ Ho ridotto il batch size (PR più piccoli, sprint più corti o Kanban)
- ☐ Ho misurato le metriche baseline PRIMA di iniziare
- ☐ Ho misurato le metriche DOPO ogni sprint per mostrare il miglioramento

10 SCENARI PRATICI CON PERSONAGGI INVENTATI

Scenario 1 — Lo Scrum Zombie

Massimo Belli, CTO di AppStack (25 developer, "facciamo Scrum da 2 anni")

Massimo ha introdotto Scrum 2 anni fa. Il team fa tutti i meeting: standup, planning, review, retro. Ma la velocity è calante, i rilasci sono in ritardo, e il morale è basso. La tua diagnosi: Scrum Zombie — i riti ci sono ma non c'è il valore. I daily standup sono status meeting. Le retrospettive producono azioni mai eseguite. Il planning non ha sprint goal. L'intervento: "Scrum Reset" — 1 sprint in cui si fa solo Scrum "minimale" (goal + daily + retro con azioni reali) per ri-acquisire il senso del perché. Poi si reintroducono i riti uno alla volta, con intento esplicito.

Scenario 2 — Il Kanban per il Team di Supporto

Giulia Ferretti, Operations Manager di ServiceDesk (8 tecnici di supporto)

Il team di Giulia gestisce 50-60 ticket al giorno con priorità che cambiano ogni ora. Hanno provato Scrum ma gli sprint non avevano senso: le emergenze arrivavano sempre nel mezzo dello sprint e rompevano tutto. La soluzione: Kanban puro con SLA espliciti per tipo di ticket (P1: 4 ore, P2: 1 giorno, P3: 3 giorni), WIP limit per tecnico (max 3 ticket in Progress), daily stand-up di 10 minuti solo sui P1 attivi. Dopo 1 mese: SLA rispettati al 92% (prima: 65%), backlog visibile, team meno stressato.

Scenario 3 — Il Product Owner Assente

Lorenzo Conti, PM di ProductCo che fa il PO part-time

Lorenzo è product manager per 3 prodotti diversi. Il team Scrum lo vede 30 minuti a sprint planning, poi scompare. Il backlog non è mai aggiornato, le user story mancano di criteri di accettazione, e il team prende decisioni di prodotto da solo (spesso sbagliate). La tua diagnosi: il problema non è tecnico, è di governance del prodotto. L'intervento: workshop con l'AD per capire le priorità del prodotto e decidere se Lorenzo può fare davvero il PO o se serve qualcuno dedicato. Risultato: assunzione di un junior PO che fa il lavoro operativo, con Lorenzo come stakeholder/reviewer. Il team finalmente sa cosa costruire.

Scenario 4 — Il DevOps da Zero

Anna Mori, CTO di StartPay (fintech, 8 developer, nessuna CI/CD)

Anna deploia con FTP su un server VPS. Ogni deploy è manuale, richiede 1-2 ore, e si fa di venerdì (perché "se va male il weekend lo risolviamo senza clienti"). Test: 0%. Nessun monitoring. Anna sa che è rischioso ma non sa da dove partire. Il tuo piano: prima i test (2 settimane per portare coverage a 30% sui flussi critici), poi la pipeline CI (GitLab CI base: build + test automatici), poi il deploy automatizzato in staging, poi in produzione con approval manuale. In 6 settimane: deploy da 2 ore a 8 minuti, da venerdì a "quando vogliamo", 0 incidenti nei 3 mesi successivi.

Scenario 5 — La Retrospettiva che Non Cambia Niente

Davide Neri, Scrum Master di DevTeam (team da 6)

Le retrospettive di Davide producono sempre le stesse 3 liste. Le azioni vengono dimenticate. Il team è stanco e inizia a saltare le retro ("tanto non serve"). Il tuo intervento: introduci il formato "What Went Well / Even Better If" invece di Keep/Drop/Try, aggiungi un "Action Board" visibile in ufficio con gli item aperti, e inizia ogni retro con "revisione delle azioni della retro precedente". In 3 sprint: il team vede che le cose cambiano, le retro diventano utili, l'engagement ritorna.

Scenario 6 — Il Team che Odia la Stima

Matteo Villa, Tech Lead di StudioDev (team da 8)

Il team di Matteo odia stimare in story point. Le discussioni durano ore. Le stime sono sempre sbagliate. Il team pensa che "la stima è inutile". La tua diagnosi: il problema non è la stima in sé, è che le user story non sono abbastanza chiare per essere stimate credibilmente. Quando stimi qualcosa di vago, la stima è inevitabilmente sbagliata. Soluzione: Definition of Ready più rigida (criteri di accettazione obbligatori prima della stima), e stime in T-shirt size (XS/S/M/L/XL)

invece di story point per ridurre il falso senso di precisione. Le storie grandi vengono spezzate prima di entrare in sprint.

Scenario 7 — Il CI/CD Che Nessuno Usa

Sara Conti, DevOps Engineer di CloudDev

Sara ha costruito una pipeline CI/CD complessa e potente. Ma i developer continuano a fare push diretti sul branch main, bypassando la pipeline. La pipeline è vista come "slow" (12 minuti) e "bloccante" (fallisce per test fragili). La tua diagnosi: una pipeline non usata è peggio di nessuna pipeline (crea l'illusione di sicurezza). Il tuo piano: 1) riduci i false positive della pipeline (i test fragili si rompono non per bug ma per flakiness — fixali o rimuovili), 2) riduci il tempo (da 12 a 5 minuti parallelizzando), 3) branch protection rule che impedisce il merge senza pipeline verde. Dopo 2 mesi: la pipeline è verde il 95% delle volte, i developer la rispettano.

Scenario 8 — L'Agile alla Grande Impresa

Roberto Amato, Head of Engineering di InsureTech (200 persone, 15 team)

Roberto deve scalare Agile da 3 team a 15. Sta valutando SAFe. Il tuo assessment: SAFe è overkill per la fase attuale. I problemi reali sono: coordinamento tra team su dipendenze tecniche, allineamento delle priorità di prodotto tra i team, e deployment non sincronizzati. Proposta: LeSS (Large-Scale Scrum) invece di SAFe — meno overhead, più focalizzato sul prodotto. Oppure, senza framework scalato, un Communities of Practice per allineare le pratiche e un Product Roadmap Review bimestrale per allineare le priorità. I team rimangono autonomi ma coordinati.

Scenario 9 — Il Manager che Vuole Kontrolle

Claudio Ricci, Project Manager di AgencyDev (15 developer)

Claudio ha approvato Agile ma continua a chiedere report settimanali su "chi sta lavorando su cosa e quante ore". Il team sente di essere controllato, non supportato. La tua diagnosi: Claudio ha paura di perdere visibilità. Il tuo lavoro non è convincerlo che i report sono inutili, ma dargli visibilità in modo Agile: board visuale accessibile a tutti, sprint review settimanale con demo dal vivo, metrics dashboard (velocity, cycle time, open bugs). Claudio ottiene visibilità reale sul progresso senza surveillance individuale. Fiducia gradualmente costruita nel tempo.

Scenario 10 — Il Case Study di Successo

Elisa Marini, CEO di TalentFlow (SaaS HR, 15 developer)

Elisa ti ha ingaggiato per un Agile Transformation Sprint. Dopo 6 settimane: Kanban funzionante con WIP limits, DoD condivisa e rispettata, retrospettive che producono azioni reali, pipeline CI/CD con coverage al 70%, e Lead Time medio sceso da 12 giorni a 4 giorni. Elisa usa i numeri per un fundraising ("delivery velocity aumentata del 66%, bug critici -80%"). Diventa un case study che porti in ogni pitch successivo.

GLOSSARIO AGILE & DEVOPS

Agile Theatre — Adozione dei rituali Agile senza adottare i valori. Il team fa gli sprint ma non ha cicli di feedback reali.

Burndown Chart — Grafico che mostra il lavoro rimanente nello sprint nel tempo. Utile per la previsione, non per il controllo.

DORA Metrics — Quattro metriche di performance Engineering validate da ricerca (DevOps Research & Assessment): Deployment Frequency, Lead Time for Changes, Change Failure Rate, MTTR.

Feature Flag — Meccanismo per attivare/disattivare una feature in produzione senza deploy. Separa il deploy dalla release.

Flaky Test — Test che passa e fallisce in modo non deterministico. Distrugge la fiducia nella pipeline. Va fixato o

rimosso immediatamente.

GitOps — Pratica in cui l'infrastruttura e il deployment sono gestiti attraverso git. Il repository è la single source of truth.

Infrastructure as Code (IaC) — L'infrastruttura è descritta in codice (Terraform, Ansible) invece che configurata manualmente. Versionabile, reproducibile, revisionabile.

Kaizen — Termine giapponese per "miglioramento continuo piccolo e incrementale". Il principio alla base delle retrospettive.

Mob Programming — L'intero team lavora insieme sullo stesso codice, stesso schermo. Estremamente efficace per knowledge transfer e per problemi complessi.

Scrum of Scrums — Meeting di coordinamento tra più team Scrum. Un rappresentante per team, focus sulle dipendenze e sui blocchi cross-team.

Sprint Zero — Sprint iniziale usato per setup del progetto (environment, backlog iniziale, DoD, working agreement). Non produce feature.

Technical Debt — Costo futuro derivante da scorciatoie tecniche prese per velocità nel presente. Va gestito attivamente (budget di refactoring in ogni sprint).

WIP (Work In Progress) — Tutto il lavoro iniziato ma non completato. Ridurre il WIP aumenta il flusso e la prevedibilità.

TOKEN & COSTO STIMATO

Voce Valore	
Lunghezza documento	~1.000 righe, ~9.500 parole
Token input stimati	~4.000 (contesto + sistema)
Token output stimati	~11.500
Modello	Claude Opus (Bedrock)
Costo output stimato	~\$0.58
Costo totale sessione	~\$0.68

I costi sono stime basate su tariffe pubbliche Anthropic/AWS Bedrock a maggio 2026.